

Programming Assignment 1 – Computer Networks

Teachers: Fatima Zahra Errounda, Stephan Schiffel

August 17, 2026

Question	Points	Score
1	35	
2	65	
Total:	100	

Logistics

- This is a group assignment, you can work in groups of up to 3 students. We highly recommend that every student in the group does all non-programming tasks individually, but we only want one submission per group.
- Answers to questions must be submitted as a pdf, with accompanying code in a zip file, using Gradescope.
 - **Source code for programming assignments must be submitted in a single zip file, with accompanying build instructions that work. Anything else will receive an automatic 0.**
 - In particular:
 - * All code must be written in C or C++
 - * The zip file uncompresses into its own directory
 - * A README file is included explaining how to compile and run the submitted code.
 - * IDE's will not be accepted
 - * Code is clearly structured and well commented. Each function/class has a header describing its purpose.
- Practical programming exercises may be done on your local laptop, or using your account on the TSAM server(130.208.246.98). Part of this assignment is getting your programming environment setup for the rest of this course. We strongly recommend that you create a suitable environment on your laptop or other machine which you can use to run client and monitoring software such as tcpdump or wireshark. If you have any issues at all in getting setup, please contact us **immediately**.

Tasks

The goal of this exercise is to first introduce you to some useful network command line tools, which can help you explore and debug network programs, and then to get you to write your first network program.

1. (35 points) **Always check the cables**

Before we start programming, let's just check that there is actually a network connection to program with. For reasons that you will learn during this course, this is not something that should ever be taken for granted. For this question we will be using a server in the cybersecurity lab with IP address 130.208.246.98 (TSAM server).

(a) (5 points) First test basic network connectivity. Use the ping command:

```
ping 130.208.246.98
```

Now check for open ports. Using the nmap command:

```
nmap -sS 130.208.246.98
```

list all open TCP ports on the TSAM server and provide a screen shot of the output of these two commands.

(b) (10 points) Now we will use ncat to set up a simple TCP client and server and use tcpdump (or wireshark) to observe the network communication between client and server.

You need to follow these steps in the right order:

1. Open 3 terminal windows side-by-side. For reference we will call them *client*, *server*, and *tcpdump*.
2. Pick a port in the range 5000-5200, that did **not** get reported by nmap as already open. We will use 5000 here in the example.
3. In the *tcpdump* window, run the command:
sudo tcpdump -vAX -i ens192 'port 5000'
Important: You may need to replace ens192 with your local machine's network interface name. You can find this with the ifconfig (OSX), ipconfig (windows), ip addr (linux) command. Hint, look at RX and TX packet counts.
4. In the *server* window, use ssh to login to the TSAM server and run the command **ncat -ln 5000**.

5. In the *client* window, run the command **ncat 130.208.246.98 5000** and type "Remember to check for network connectivity" followed by return.

The text should now appear in the server window and tcpdump should show some captured packets including one with the text you typed.

Submit a screenshot of the *tcpdump* window that shows all the captured packets.

- (c) (10 points) In your own words, explain what each of the three commands does. Hint: it may help to read the manual (man page) for ncat and tcpdump.
- (d) (5 points) Repeat (b) above, but use UDP instead of TCP. (Hint: You will have to change the ncat commands.)
- (e) (5 points) How does the tcpdump output differ when you used UDP vs TCP? Why?

Do not leave any of these processing running on the TSAM server (or your local machine) after you are done!

2. (65 points) **UDP Port Scanner**

Write a UDP port scanner, that takes in as arguments the IP address of the machine, and a range of ports to scan between. The scanner should be run with the command:

```
./scanner <IP address> <low port> <high port>
```

To do this, you will have to:

- program with the socket API (using `socket()`, `send_to()`, `recv_from()`, etc)
- set up a UDP client socket (using `socket()`)
- set up the server address (a structure `struct sockaddr_in`)
- to each of the ports in range, send a packet (using `send_to()`)
- wait to see if you get a response from the port (using `recv_from()`)
- report the port as open when you get a response

Note that UDP is an unreliable protocol. Some packets may be dropped randomly, which means you will have to repeat your packets multiple times to make sure the packet or response was not dropped. Also, note that UDP is not connection-oriented: There is no guarantee that a packet you receive is a response to the packet you sent. It may come from someone else.

Use your scanner to scan UDP ports between 4000-4100 on the TSAM server and print out the open ports that you find in this range.

You can compare your results to a scan using `nmap` (with appropriate parameters to scan UDP ports in the given range) to see whether you get the same result.

Useful tools

The following table is a summary list of command line tools that can be used for networking purposes. In particular, nmap, ncat and tcpdump, will be useful for this assignment. In general, **wireshark** (or tcpdump) is very useful for any kind of debugging of network programs. Some of these commands may need to be installed using the package management for your local machine, and should also be available via the bash console on Windows.

Useful Network Commands (Linux)			
man	Online manual for command	man -k	keyword search on manual
whois	Domain registry information	htop	Enhanced top, show processes
iftop	Show top traffic/network	iptraf-ng	IP traffic monitor
route	show local routing	arp -v	Show address resolution cache
ncat <ip> <port>	Connect to remote system	nmap	Scan remote host
netstat	Show network statistics	netstat -antup	Process path info
netstat -t	Show only tcp connections	netstat -u	show only udp connections
ss	Show detailed network info	ss -s	Summary of network info
ifconfig	Network Interface	ip	Enhanced ipconfig/network info.
dig	DNS record lookup	dig <domain>	Get full record for <domain>
nmap	scan host ports	wireshark	GUI version of tcpdump
tcpdump	inspect network traffic		
nslookup	interactively query DNS	set type=A	Specify A records
		server=	Specify server to query